



HW4

QR Code Decoder

簡毓寬

2025/10/23

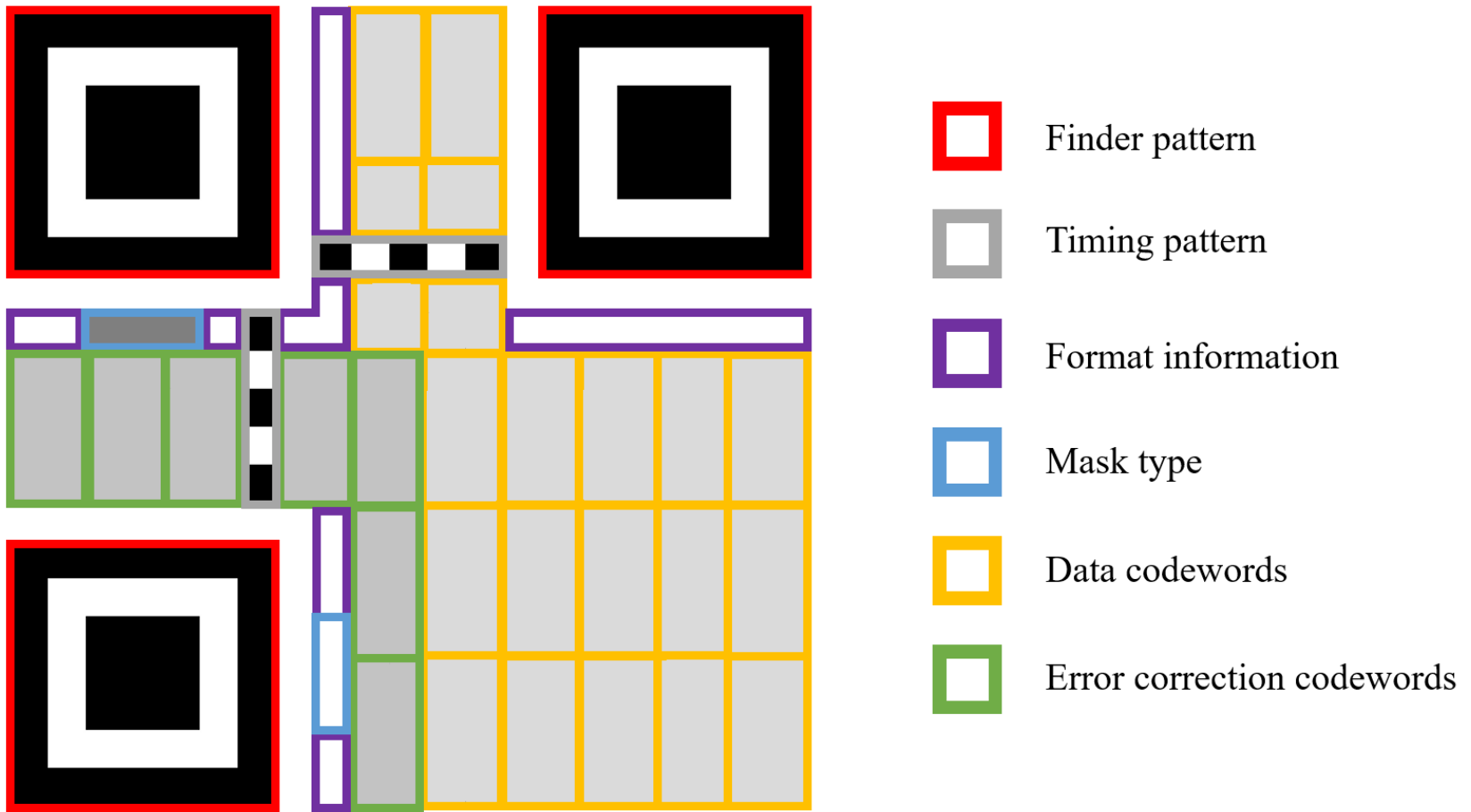
Introduction

14-byte “www.google.com”



- QR Code
 - Quick Response Code
 - Widely used for URL (Uniform Resource Locator) instant access 網址
 - A variety of spec
 - Version 1 to 40 (from 21x21 to 177x177 pixels)
 - Different character set
 - Different level of error correction capability
 - Spec used in this assignment
 - Version 1 (**21x21** pixels)
 - 8-bit Byte mode with ISO 8859-1 code
 - Error correction level L (7% recovery capacity)
 - In this assignment, the codewords are error free

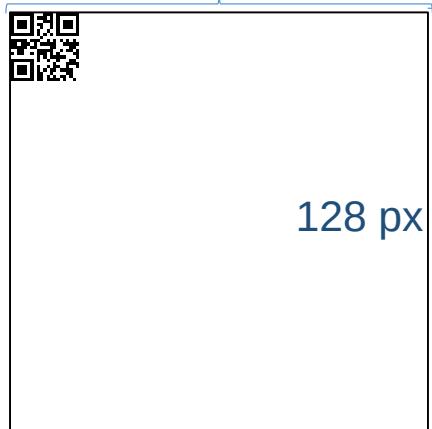
Structure of version 1-L QR Code



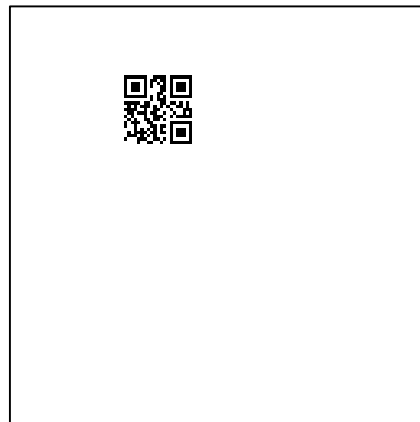
Four ranks of test pattern

Pattern	# of QR Code in a pattern	Location	Rotation	Scale	Background
Rank A	Arbitrary (1 to 8)	Arbitrary	4 way	1x / 2x	Yes
Rank B	Arbitrary (1 to 8)	Arbitrary	4 way	1x	Yes
Rank C	1	Arbitrary	4 way	1x	No
Rank D	1	Upper-Left	0-degree	1x	No

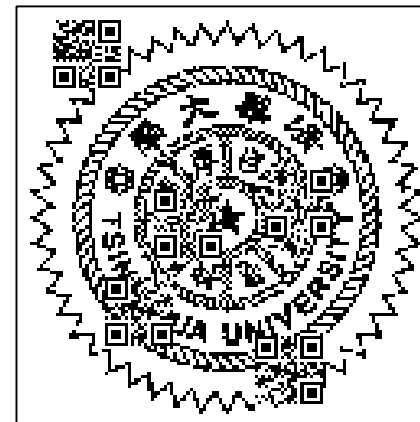
128 px



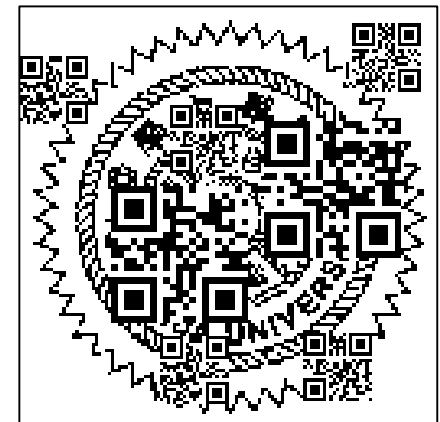
Rank D



Rank C

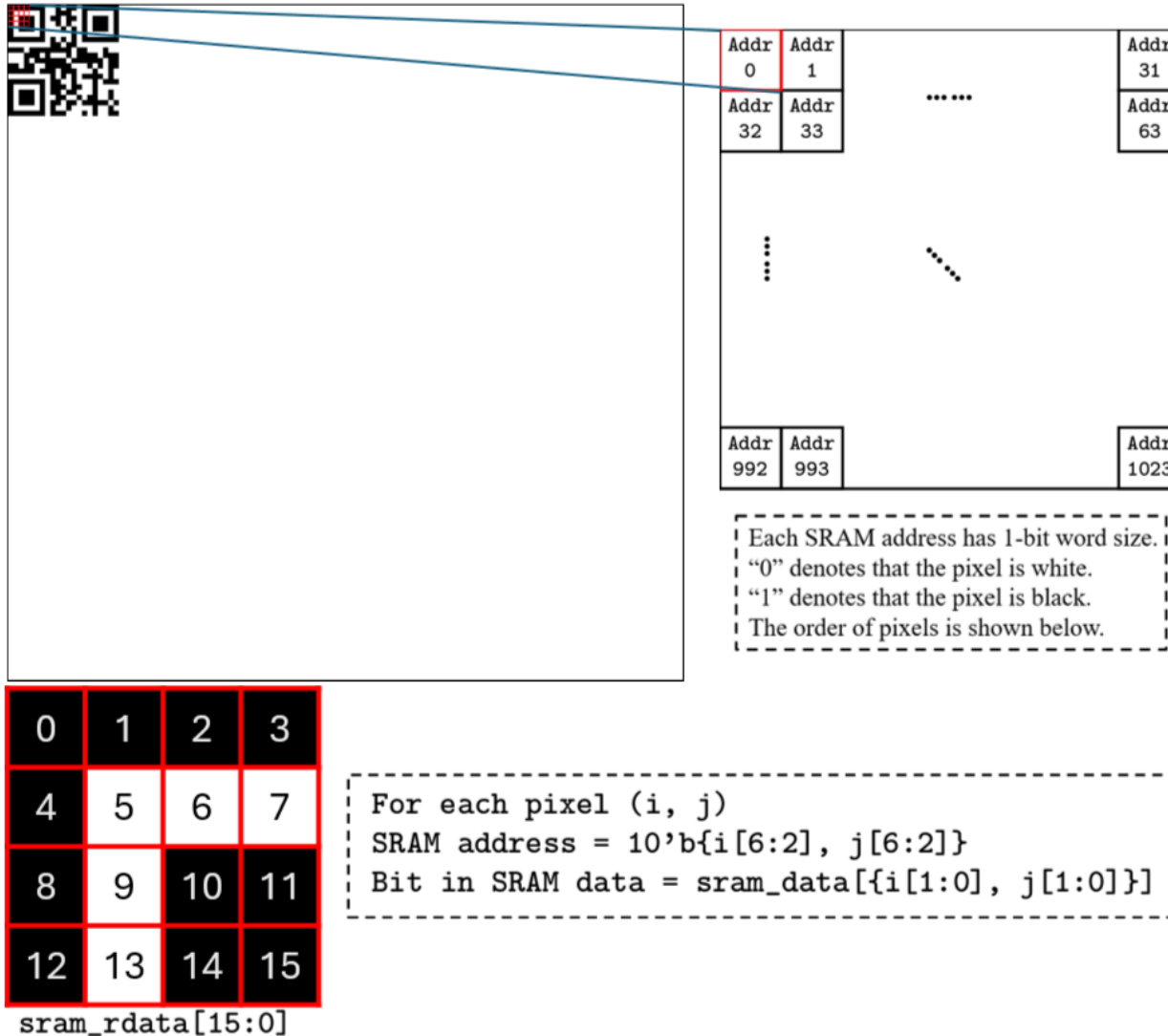


Rank B

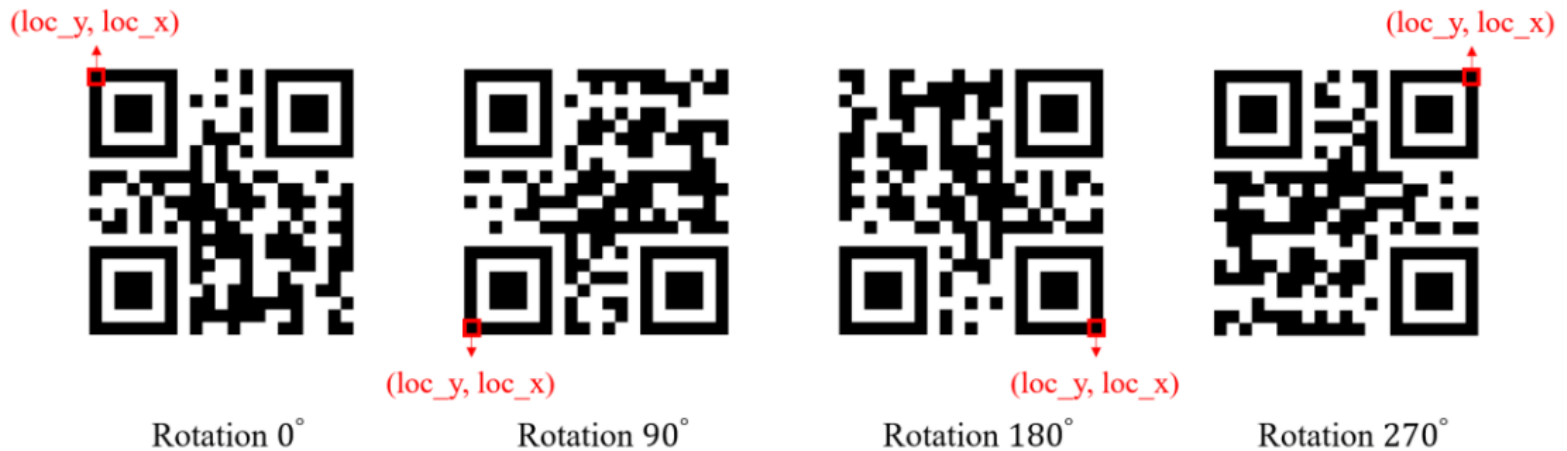
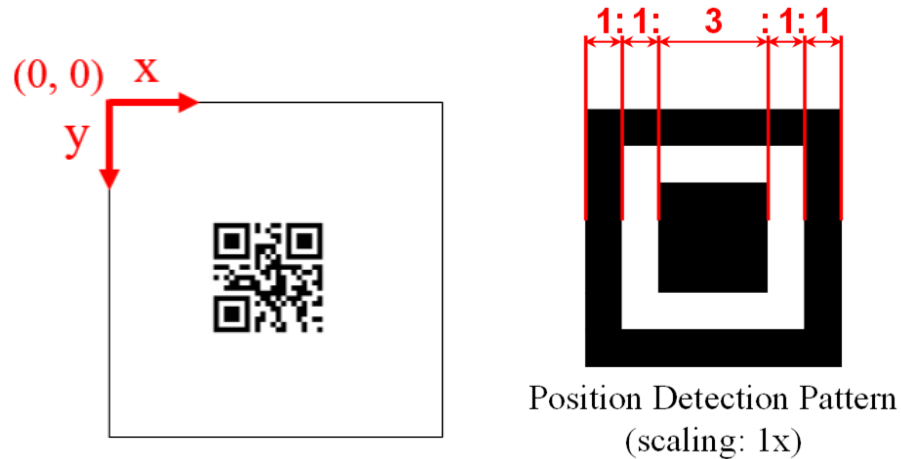


Rank A

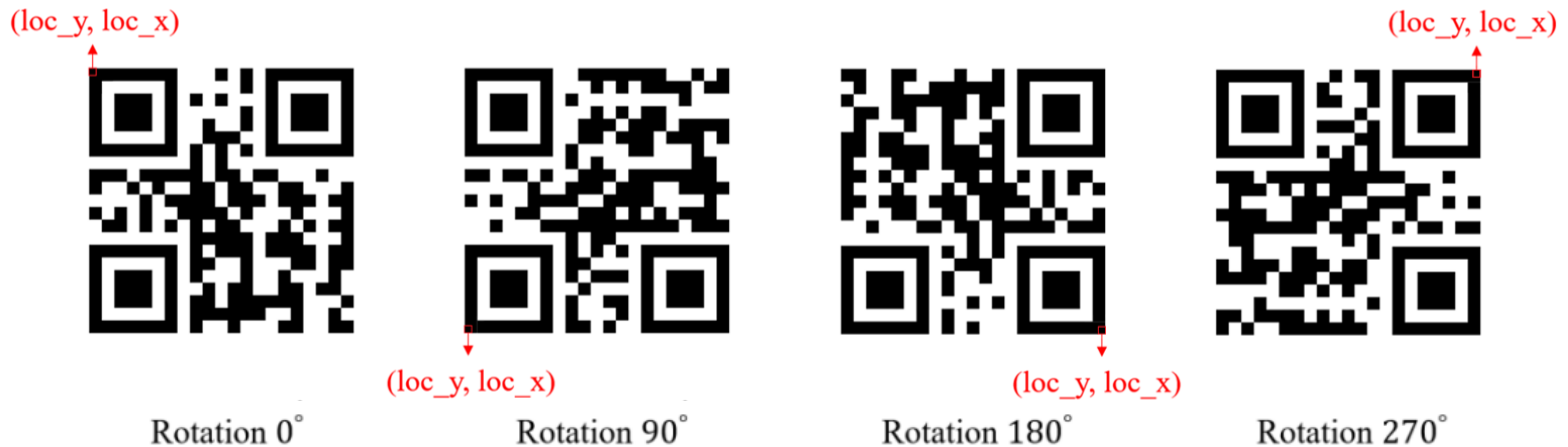
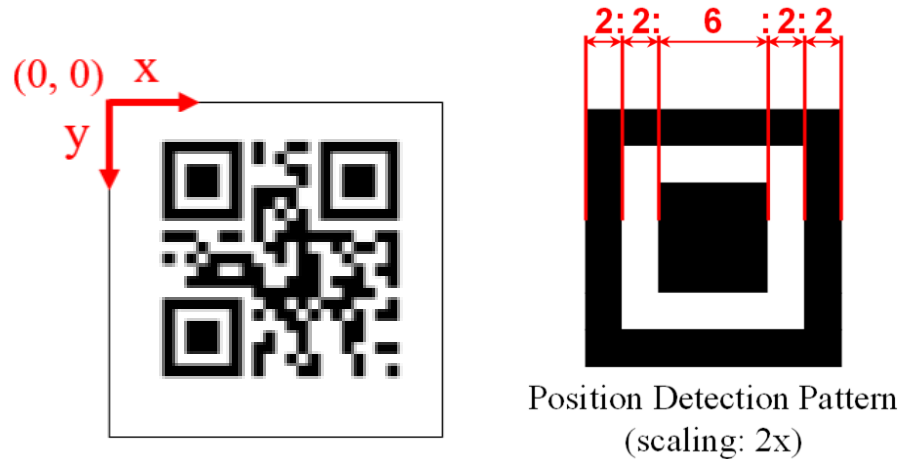
Data arrangement in SRAM



Rotation and Location of QR code (1x)

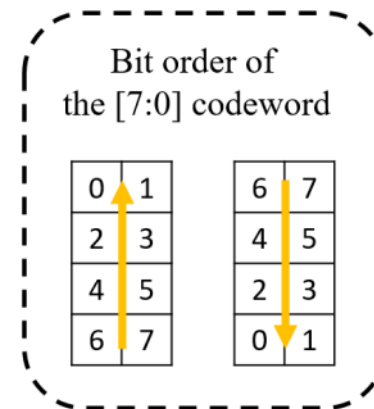
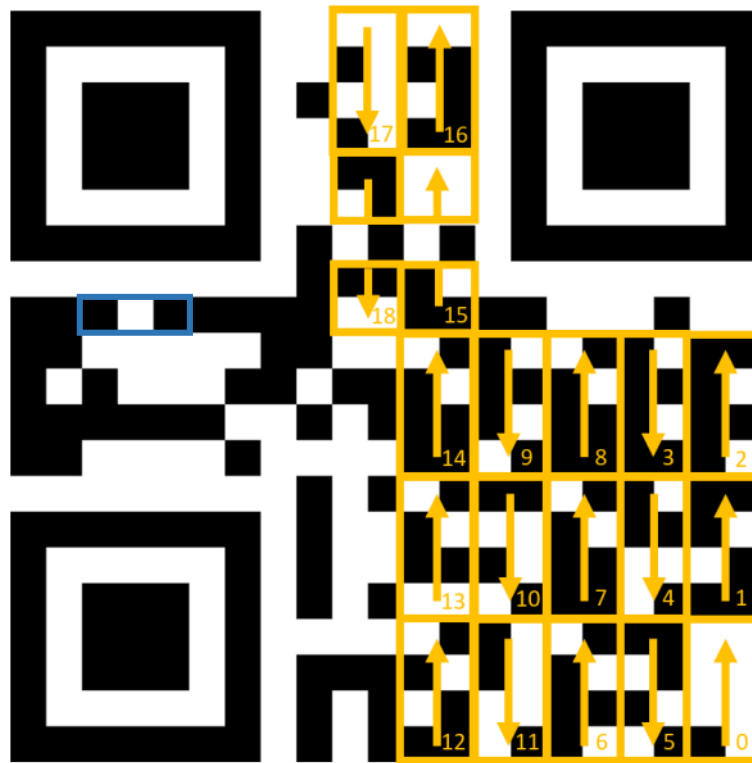


Rotation and Location of QR code (2x)



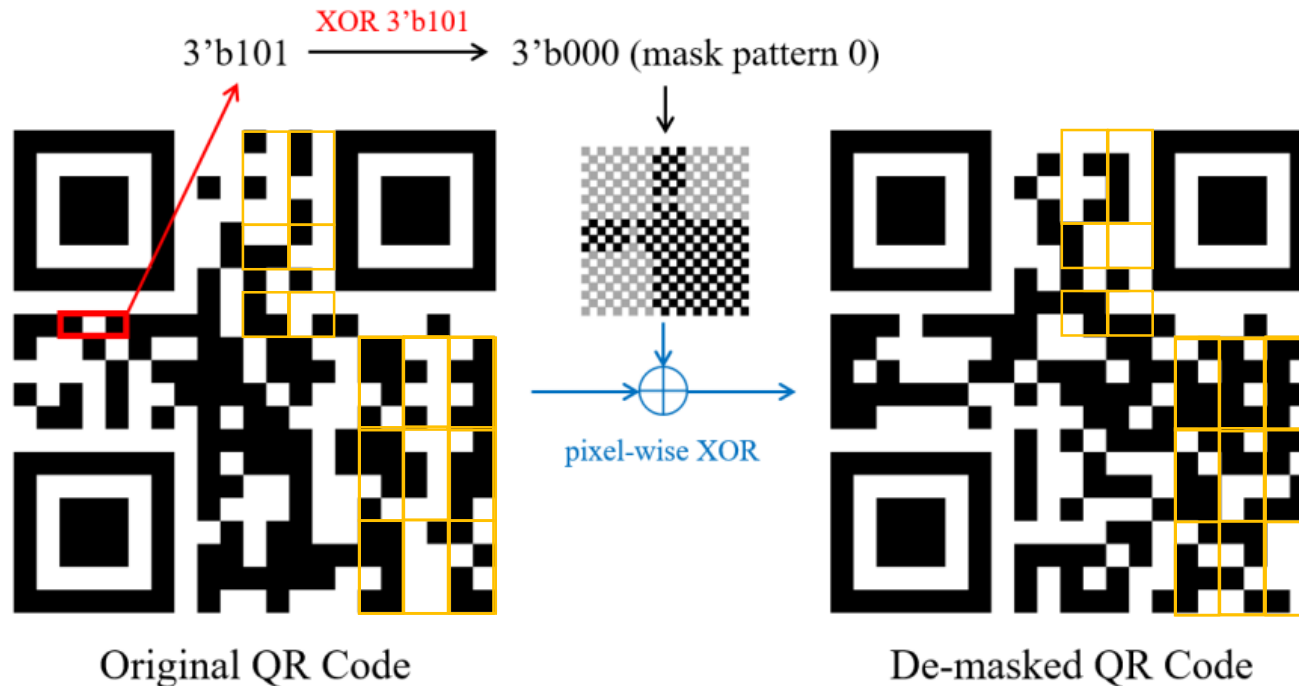
Decode the data codewords

In this homework, the QR code is error-free



De-masking process

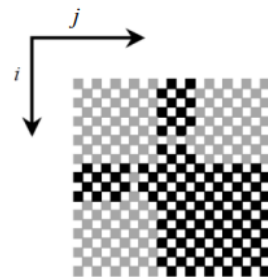
- The 3-bit data should first be XORed with a fixed 3'b101 to derive the real mask ID



De-masking process

- Eight kinds of mask pattern

Mask Pattern Reference	Condition
000	$(i + j) \bmod 2 = 0$
001	$i \bmod 2 = 0$
010	$j \bmod 3 = 0$
011	$(i + j) \bmod 3 = 0$
100	$((i \div 2) + (j \div 3)) \bmod 2 = 0$
101	$(i \div j) \bmod 2 + (i \div j) \bmod 3 = 0$
110	$((i \div j) \bmod 2 + (i \div j) \bmod 3) \bmod 2 = 0$
111	$((i \div j) \bmod 3 + (i \div j) \bmod 2) \bmod 2 = 0$



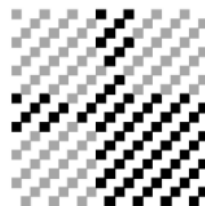
000
 $(i + j) \bmod 2 = 0$



001
 $i \bmod 2 = 0$



010
 $j \bmod 3 = 0$



011
 $(i + j) \bmod 3 = 0$



100
 $((i \div 2) + (j \div 3)) \bmod 2 = 0$



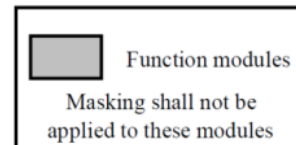
101
 $(i \div j) \bmod 2 + (i \div j) \bmod 3 = 0$



110
 $((i \div j) \bmod 2 + (i \div j) \bmod 3) \bmod 2 = 0$



111
 $((i \div j) \bmod 3 + (i \div j) \bmod 2) \bmod 2 = 0$



Decode the text sequentially

- Data arrangement in data codewords
 - The text length information tells you how many characters should be decoded
 - Once decoding a text data, you can put it to the output port $[7:0]$ *decode_text* and set the output port *valid* to high

Codeword 0		Codeword 1		Codeword 2		Codeword 3		...
0	100	0000	1111	0110	0001	0110	0010	0111
Data encoding type (4'b0100)		Text length (8'b00001111)		Text 0 (8'b01100001)		Text 1 (8'b01100010)		

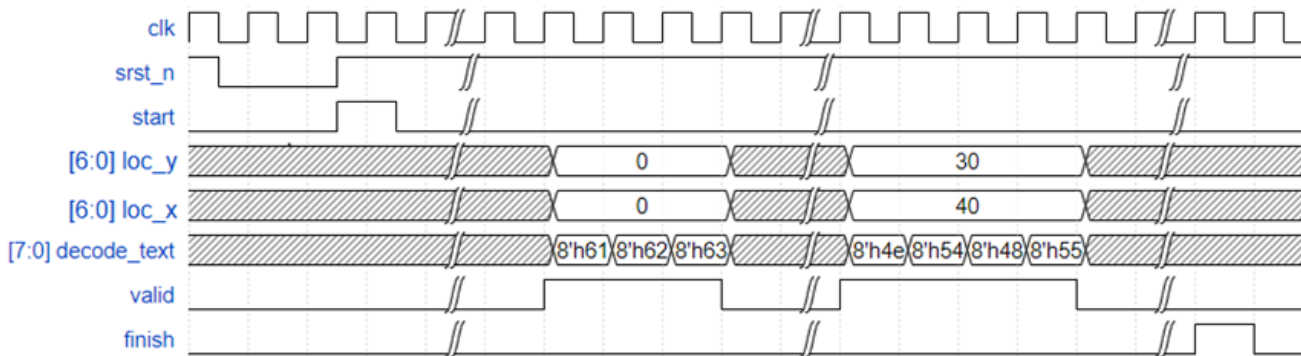


Type	Name	Number of bits	Description
Input	clk	1	Clock
Input	srst_n	1	System reset (synchronous, active low)
Input	start	1	Indicate that you can start decoding (one-cycle pulse)
Input	sram_rdata	16	SRAM read data
Output	sram_raddr	10	SRAM read address
Output	loc_y	7	Row location of the QR Code
Output	loc_x	7	Column location of the QR Code
Output	decode_text	8	Decoded text
Output	valid	1	Indicate that the decode_text is valid when it is high
Output	finish	1	Set high after all QR Codes are decoded



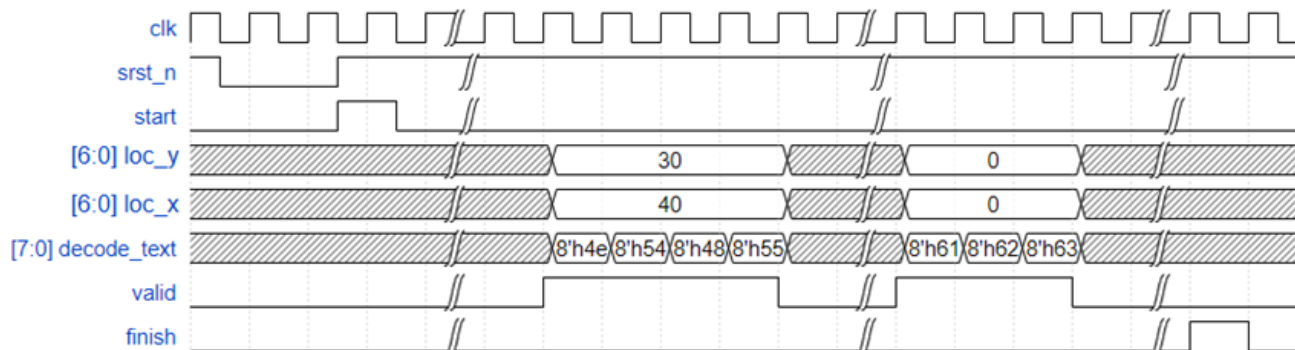
Timing diagram

- Assume two QR codes in a pattern
 - One at (0, 0), {8'h61, 8'h62, 8'h63} which represents “abc”
 - One at (30, 40), {8'h4e, 8'h54, 8'h48, 8'h55} which represents “NTHU”



**QR Code at (0, 0)
is output first**

The output order of
QR Codes is not limited

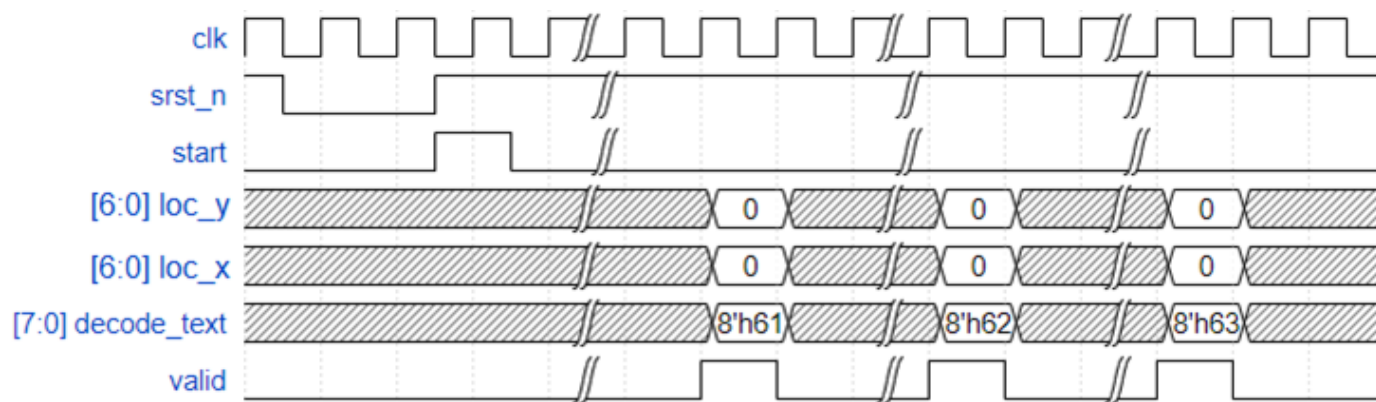


**QR Code at (30, 40)
is output first**



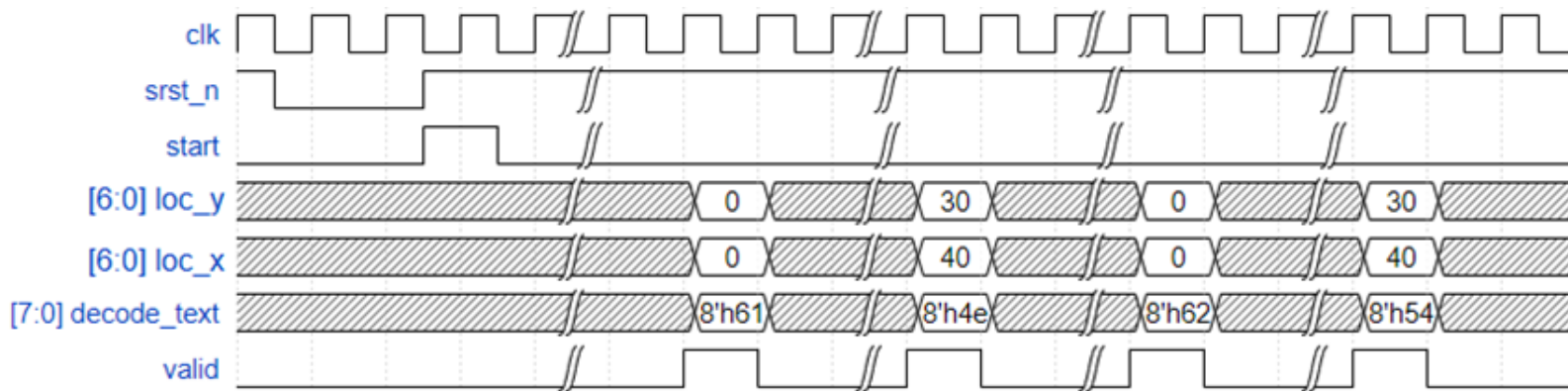
Timing diagram

- The output *decode_text* is not mandatory to be consecutive
- But the order of the output *decode_text* should still be correct
 - If the text is “abc”, you should first output “a”, and then “b”, and then “c”



Timing diagram

- You can only output the decoded result of the next QR Code after all the decoded result of the previous one have been output
- Below is **Not** acceptable output behavior



Simulation

- For Rank A, there are total 300 patterns
- For Rank B, Rank C, and Rank D, there are 200, 180, 60 patterns for each Rank respectively
- To pass each Rank, your design should pass all patterns in the Rank

```
# # Rank A
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_A+DEBUG_MODE+PAT_L=0+PAT_U=299

# # Rank B
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_B+DEBUG_MODE+PAT_L=0+PAT_U=199

# # Rank C
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_C+DEBUG_MODE+PAT_L=0+PAT_U=179

# Rank D
vcs -f hdl.f -full64 -R -debug_access+all +v2k \
+define+RANK_D+DEBUG_MODE+PAT_L=0+PAT_U=59
```

For debugging convenience, you can modify **PAT_L** and **PAT_U** to simulate with any specific range of patterns

Synthesis

- For fairness, please do logic synthesis with the provided scripts
- Only two parts of the scripts can be modified
 - line 16 in 0_readfile.tcl
 - Add your hdl files
 - line 5 in synthesis.tcl
 - Change your clock timing constraint

```
12 # Define a lib path
13 define_design_lib $TOPLEVEL -path ./$TOPLEVEL
14
15 # Add your hdl files here
16 analyze -library $TOPLEVEL -format verilog "../hdl/qr_decoder.v"
17
18 # Elaborate your design
19 elaborate $TOPLEVEL -architecture verilog -library $TOPLEVEL
20
```

0_readfile.tcl

```
1 # Set your TOPLEVEL here
2 set TOPLEVEL "qr_decoder"
3
4 # Change your timing constraint here
5 set TEST_CYCLE 10.0
6
```

synthesis.tcl

Grading Policy

- Functionality (60 points)
 - Rank A (20 points)
 - Rank B (15 points)
 - Rank C (15 points)
 - Rank D (10 points)
 - Note that your RTL must pass spyglass (synthesizable)
- Logic synthesis
- PI ranking (40 points)
 - 5 points if you fill in PI correctly in the PI ranking list
 - 35 points PI ranking **if your $PI \leq \text{threshold}$**
 - Gaussian-based ranking formula
 - $\text{Score} = 35\% \times (60 + 20 \times \Phi^{-1} \left(1 - \frac{\text{Rank} - 0.5}{N} \right))$ points
 - Additional 2% and 1% of the total score of the semester are given to #1 and #2

Notice

- Do not modify I/O, testbench, or patterns.
- Do not modify synthesis scripts except the two parts mentioned in the document.
- After submission, TA will use the **released version** testbench and patterns to run the simulation and check the correctness of your design.

Submission

- Deadline
 - 11/06 23:59
- Submit to eeclass
 - Make sure the file delivery and organization meet the requirement
 - Wrong file delivery or organization will get 1% punishment
- If you have any question, feel free to
 - Ask on eeclass discussion
(maybe other students also have the same question !)
 - Ask TA during the lab time or send email to TA(毓寬)
 - Do not show your code on eeclass discussion or email your code to TA. Coding and debugging by yourself !!
- Using generative AI is not recommended.
 - If used, please explain in the report how it was applied in misc.txt

